

# IT5007 Final Project Report

**Course:** IT5007 Software Engineering on Application Architecture

**Project:** CampusCart — NUS Campus Marketplace

**Group:** 14

Group number on Canvas: 14

| Student ID (AxxxxxxxZ) | NUSNet ID (exxxxxxx) | Name (as it appears on Canvas) |
|------------------------|----------------------|--------------------------------|
| A0183438X              | e0310233             | Ang Lee Chuan                  |
| A0183378R              | e0310173             | Chou Han Xian, Aaron           |
| A0329448R              | e1553775             | Liaw Jian Wei                  |

---

## Table of Contents

- [1. Problem Statement, Motivation & Competition Analysis](#)
  - [2. Solution Architecture & Design](#)
  - [3. Core Features & Implementation](#)
  - [4. Software Engineering Practices](#)
  - [5. AI Integration & Model Selection](#)
  - [6. Competition, Legal, and Business Model](#)
  - [7. Frameworks, Tooling & Developer Experience](#)
  - [8. Future Scalability & Deployment](#)
  - [Appendix A — Prototype Mockups \(Initial Figma designs\)](#)
- 

## 1. Problem Statement, Motivation & Competition Analysis

The National University of Singapore (NUS) has a large, dynamic student population with a constant need to buy and sell textbooks, electronics, furniture, and other essentials. However, there is no dedicated, trusted channel for peer-to-peer student commerce. Existing alternatives—Carousell and Telegram buy/sell groups—lack NUS verification, structured discovery, and efficient coordination, leading to trust and usability issues.

**CampusCart** was conceived to address these gaps by providing:

- **Verified NUS-only access** (via email authentication)
- **Structured, searchable listings** (categories, filters, map)
- **AI-powered listing creation and search**
- **Integrated payments (Stripe test mode)**
- **Modern, mobile-friendly UI**

**Relevance:** With 40,000+ NUS students cycling through goods each semester, the need for a safe, efficient campus marketplace will persist for years. The solution is designed to be robust and extensible for future cohorts or real-world deployment.

**Competition:**

- **Carousell:** No NUS verification, generic UI, no campus-specific features.
- **Telegram buy/sell groups:** Unstructured, no search/filter, trust issues.

- **CampusCart** differentiates via NUS-only access, AI features, and spatial discovery (interactive map, demand heatmap).

#### Legal/Business Model:

- All code is original or open source (see README for attributions). No proprietary dependencies. Project is open source and can be extended by future cohorts. No user data is collected beyond demo/test accounts. The architecture supports future commercialization or open-source community adoption.

## 2. Solution Architecture & Design

CampusCart is architected as a modern, modular full-stack web application, designed for reliability, extensibility, and a seamless developer and user experience. The architecture was informed by the initial Figma prototypes (see Appendix A), but evolved to address real-world technical constraints and opportunities for innovation.

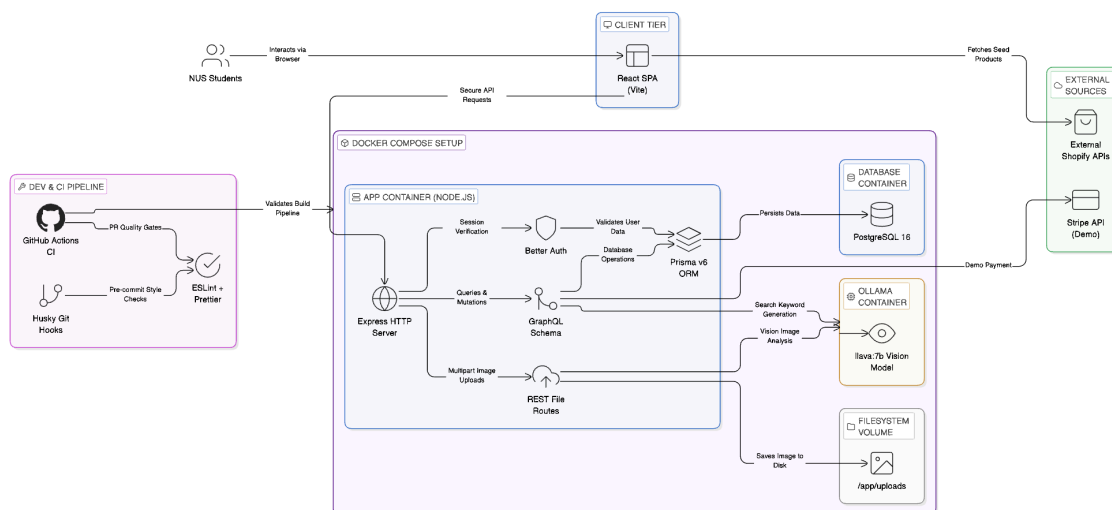


Figure 2.1: System architecture diagram

### 2.1 System Overview

- **Frontend:** Built with React (Vite) for fast development and hot module replacement. Tailwind CSS and a custom NUS design system ensure visual consistency and accessibility. React Router enables SPA navigation. Components are modularized by domain (Marketplace, Profile, Payment, etc.) for maintainability.
- **Backend:** Node.js with Express serves a GraphQL API, using `@graphql-tools/schema` for lightweight, flexible schema stitching. The backend is organized by domain (listings, requests, profile), with resolvers and schema definitions co-located for clarity.
- **Database:** PostgreSQL (Dockerized) provides a robust, scalable data store. Prisma ORM enables type-safe queries and easy schema evolution. The database is seeded with demo data for consistent evaluation and onboarding.
- **AI Integration:** A local LLM (Ollama, `llava:7b`) powers image-based autofill and search suggestions. This approach avoids external API costs and privacy concerns, while enabling advanced features like vision-based form population and semantic search fallback.
- **Payments:** Stripe integration (test mode) demonstrates secure, industry-standard payment flows using the PaymentIntent API. Sensitive keys are never exposed to the frontend, and the architecture

is ready for production upgrades (webhooks, live keys).

- **DevOps:** Docker Compose orchestrates both evaluator and developer flows. Health checks, automated seeding, and environment parity ensure a smooth experience for both instructors and contributors.

## 2.2 Key Architectural Decisions & Rationale

- **GraphQL over REST:** Chosen for its flexibility in fetching only the required data, reducing over-fetching and enabling rapid UI iteration. The lightweight schema approach avoids Apollo's complexity while retaining full GraphQL power.
- **REST for File Uploads:** GraphQL is not well-suited for binary data. By using REST endpoints (Multer) for image uploads, we keep the GraphQL layer clean and type-safe, and make it easy to swap storage backends in the future.
- **Prisma Singleton:** To avoid exhausting the PostgreSQL connection pool (a common Node.js pitfall), all resolvers share a single PrismaClient instance. This ensures efficient resource usage and reliability.
- **Session-Based Authentication:** Both frontend and backend independently validate sessions, ensuring security even if one layer is bypassed. ProtectedRoute components on the frontend and middleware on the backend enforce access control.
- **Component & Constant Sharing:** Categories and campus locations are defined in shared modules, ensuring total sync between frontend rendering and backend validation. This eliminates class-of-bugs where UI and API diverge.
- **Local LLM:** Running Ollama locally (in Docker) allows us to offer AI features without latency, cost, or privacy issues. We evaluated several models and standardized on `llava:7b` for its balance of speed and accuracy on commodity hardware.
- **Stripe PaymentIntent Model:** We adopted Stripe's recommended architecture for PCI compliance and security. The server creates PaymentIntents and only exposes client secrets to the frontend, never the secret key. This pattern is ready for production with minimal changes.

## 2.3 Design Evolution & Lessons Learned

- The initial Figma prototypes provided a strong foundation, but several features (e.g., AI autofill, map heatmap, Stripe integration) required architectural pivots and technical research.
- We prioritized developer experience: hot reload, one-command setup, and clear modularization enabled rapid iteration and onboarding.
- Security and data integrity were non-negotiable: all mutations are authenticated, and all user input is validated both client- and server-side.
- The architecture is intentionally extensible: adding new listing types, integrating new AI models, or swapping storage backends can be done with minimal refactoring.

**Appendix A** contains the original Figma prototypes that guided the UI/UX and component structure.

---

## 3. Core Features & Implementation

This section details the major features, their technical implementation, and the rationale behind key design choices. File references are provided for traceability and ease of evaluation.

### 3.1 Verified Authentication & User Management

- **NUS Email Exclusivity:** Registration is strictly restricted to `@u.nus.edu` email domains. Server-side regex validation ( `server/auth.js` ) ensures only NUS students can join. This builds trust and exclusivity.

- **Security:** Passwords must meet strict policies (min 8 chars, uppercase, number, special char). Sessions are managed via secure cookies. Both frontend and backend enforce authentication (see `src/components/ProtectedRoute.jsx` , `server/middleware/requireAuth.js` ).
- **Profile Management:** Users can edit their bio, phone, location, and upload a profile picture ( `src/pages/ProfilePage.jsx` , `server/graphql/profile/` ). Activity tracking (listings, requests) is shown on the profile page.

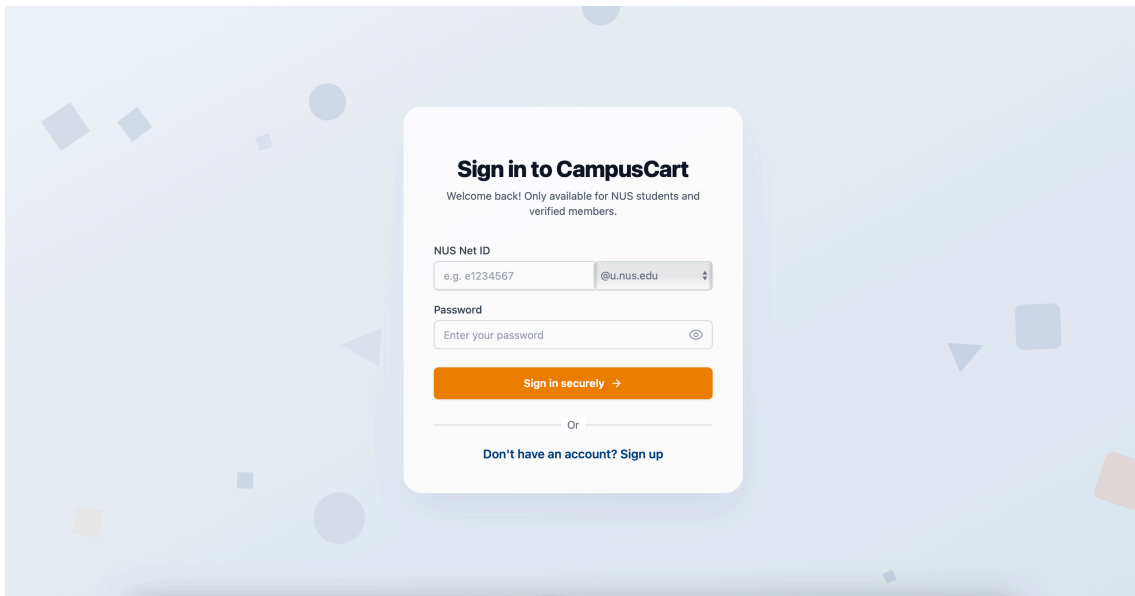


Figure 3.1.1: Login page — NUS-exclusive authentication with email domain restriction.

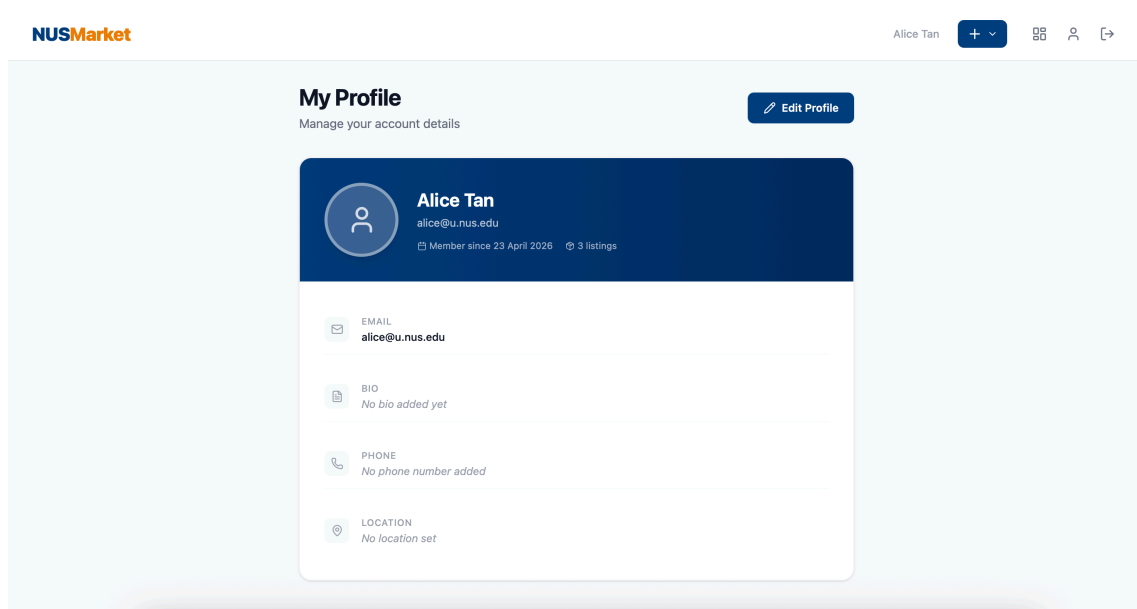


Figure 3.1.2: Account settings — user can edit profile, contact details, and preferences.

## 3.2 Marketplace (Listings)

- **Discovery & Filtering:** Buyers can filter by category, location, and seller. The sidebar and map provide advanced filtering ( `src/components/Marketplace/FilterSidebar.jsx` ,



NUSMap.jsx ).

- **CRUD Operations:** Sellers can create, edit, delete, and browse listings. Each listing includes title, description, price, condition, category, and location ( `src/pages/CreateListingPage.jsx` , `EditListingPage.jsx` , `MyListingsPage.jsx` , `server/graphql/listings/` ).
- **Image Uploads:** Listings support photo uploads (max 5MB). Images are stored on the server and served via Express static middleware ( `server/routes/upload.js` ).

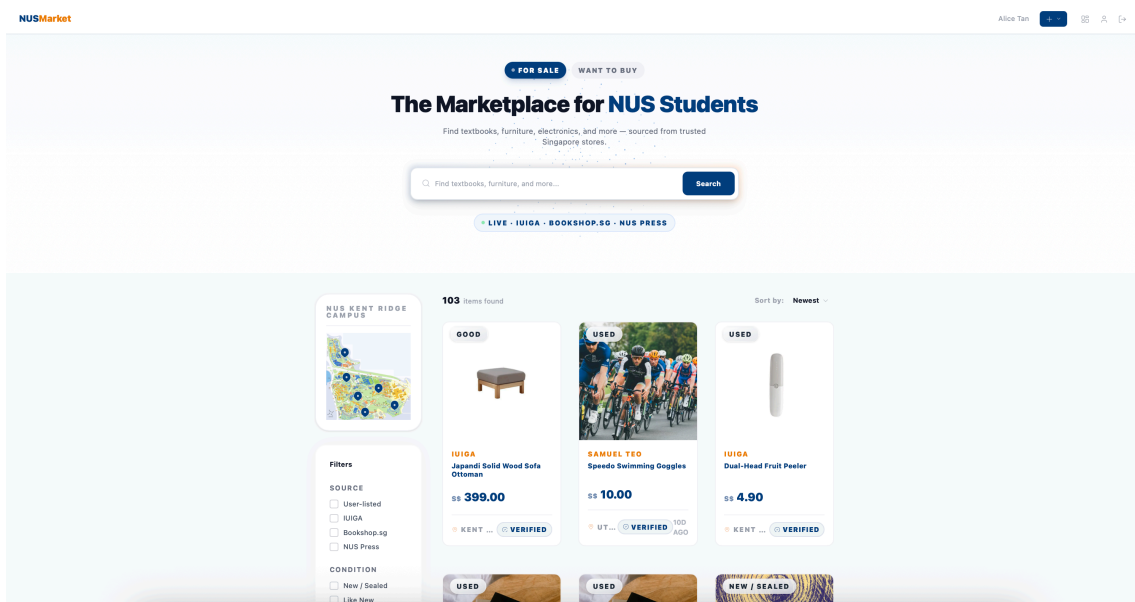


Figure 3.2.1: Marketplace homepage — listings from both external distributors and NUS users, with full filtering and search.

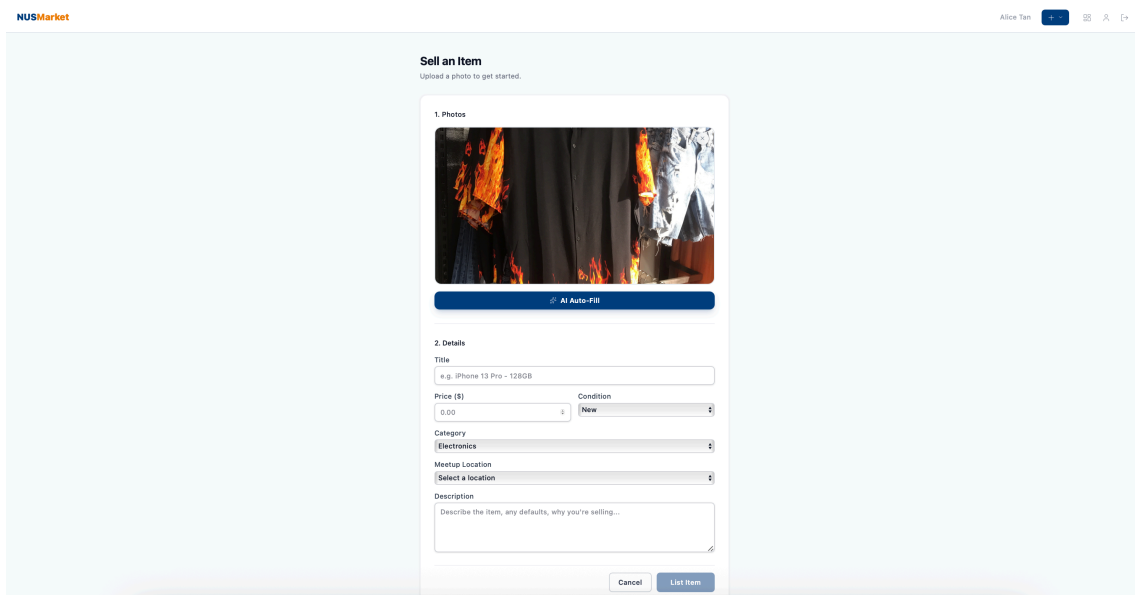


Figure 3.2.2: Create listing form — sellers enter title, description, price, condition, and upload images.

### 3.3 Requests (Want to Buy)

- **Buyer-Driven Workflow:** Students can post "want to buy" requests, specifying title, description, budget, and condition ( `src/pages/CreateRequestPage.jsx` , `EditRequestPage.jsx` , `WantToBuyPage.jsx` , `server/graphql/requests/` ).
- **Unified Navigation:** Requests and listings share the same category/location system and are accessible via dashboard and quick-action menus.

Figure 3.3.1: Create buy request form — students post items they want to purchase for others to fulfill.

Figure 3.3.2: My Listings dashboard — view, edit, or delete all your listings and requests.

### 3.4 AI-Powered Features

- **Auto-Fill:** Users can upload an item photo to auto-generate title, description, price, condition, and category using the local `llava:7b` model ( `src/components/ui/AiAutoFillButton.jsx` ,

```
server/services/ai.js ).
```

- **Search Suggestions:** When a search yields no results, the LLM suggests alternative queries ( `src/components/ui/AiSearchSuggestions.jsx` ).
- **Technical Rationale:** Local LLM avoids API costs, latency, and privacy issues. Model selection was based on speed and accuracy trade-offs.

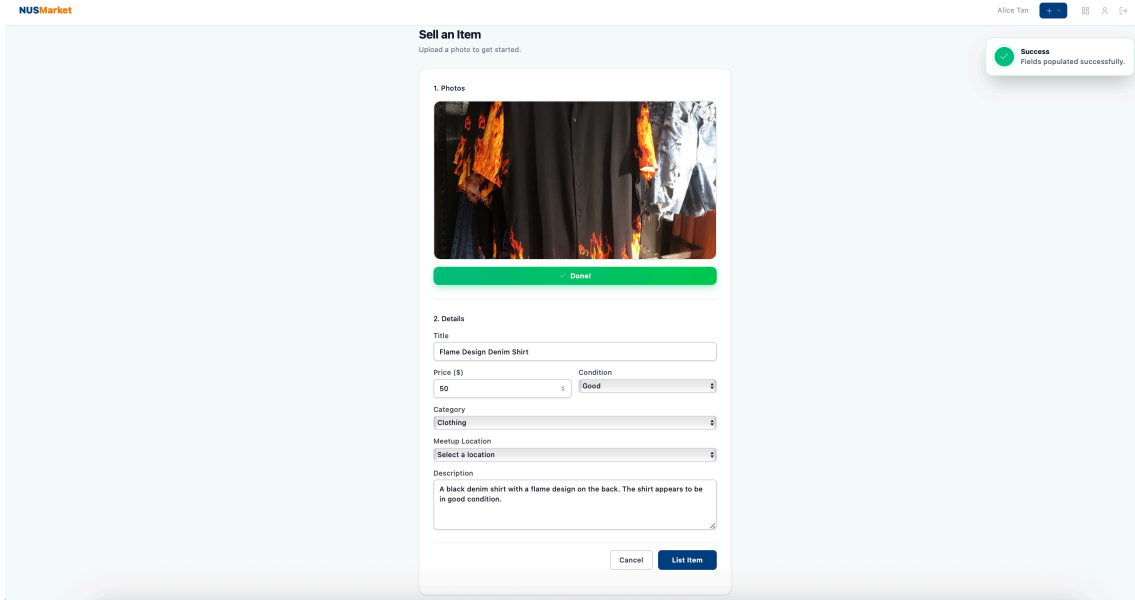


Figure 3.4.1: AI-powered auto-fill — generate listing details automatically from an uploaded image.

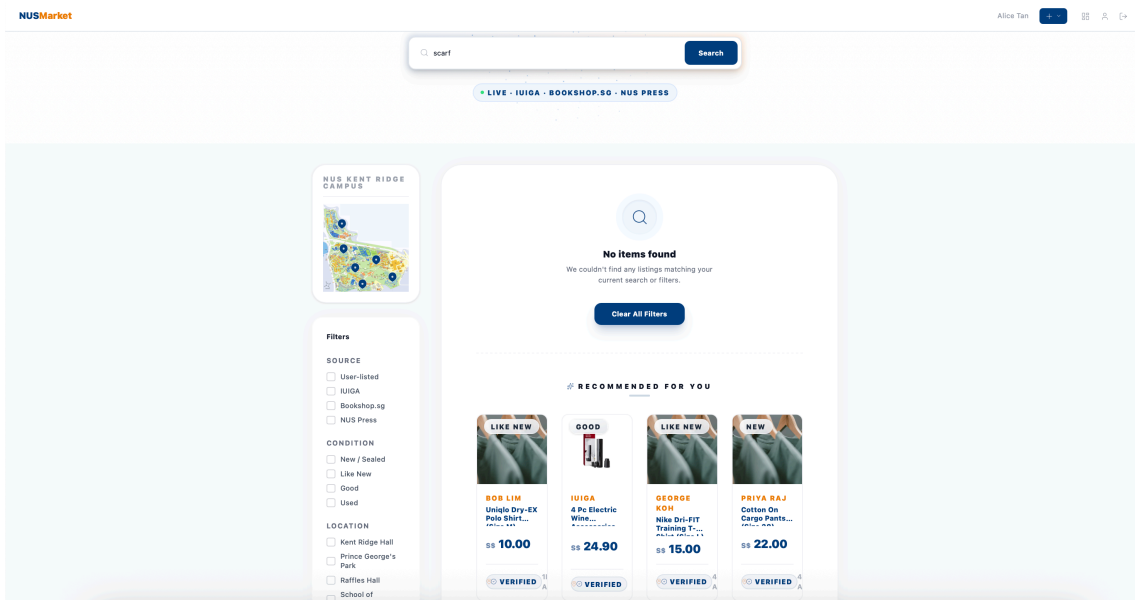


Figure 3.4.2: AI search suggestions — recommended products shown when no direct matches are found.

### 3.5 Interactive Map & Demand Heatmap

- **SVG Map:** A custom SVG map of NUS campus enables spatial filtering of listings. Pins are clickable and bi-directionally synced with sidebar filters ( `src/components/Marketplace/NUSMap.jsx` ).

- **Demand Heatmap:** The "Want to Buy" dashboard visualizes request density with a color-coded heatmap and animated hotspots ( `server/graphql/requests/requests.resolvers.js` ).
- **Design Decision:** The map enhances local discovery and provides actionable data for sellers.

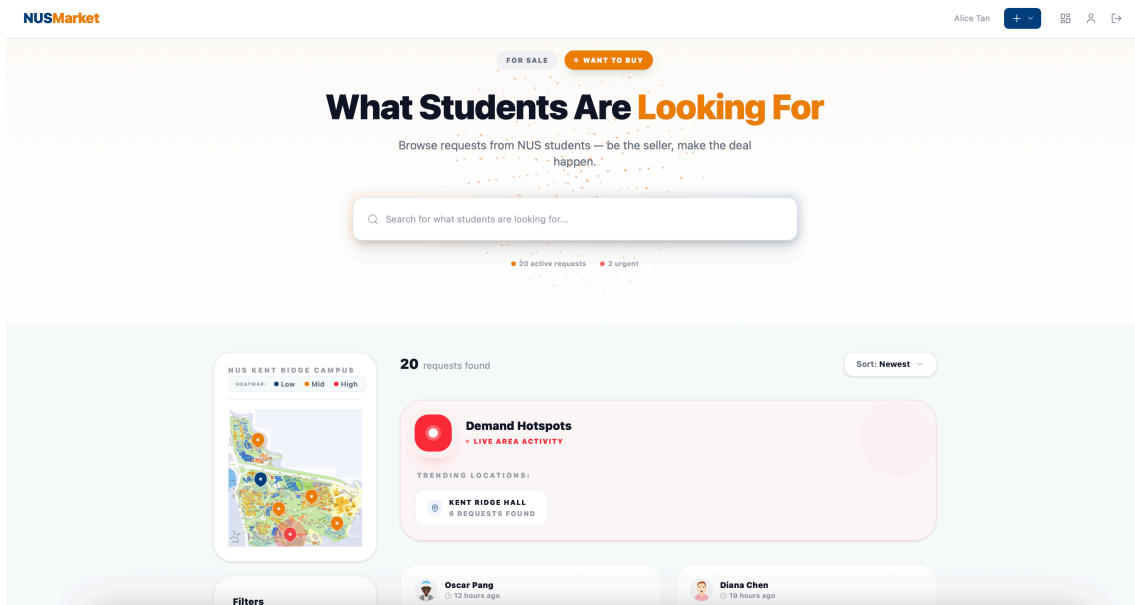


Figure 3.5.1: Demand heatmap — highlights campus locations with the most active buy requests.

### 3.6 Payments (Stripe)

- **Secure Checkout:** Stripe PaymentIntent API is used for secure, PCI-compliant payments. The server creates PaymentIntents and only exposes client secrets to the frontend ( `server/services/stripe.js` , `src/components/payment/PaymentModal.jsx` ).
- **Test Mode:** No real charges are processed. The architecture is ready for production with webhooks and live keys.
- **UX Flow:** Buyers see a two-panel modal with order summary and embedded Stripe Elements card form, styled to NUS brand tokens.

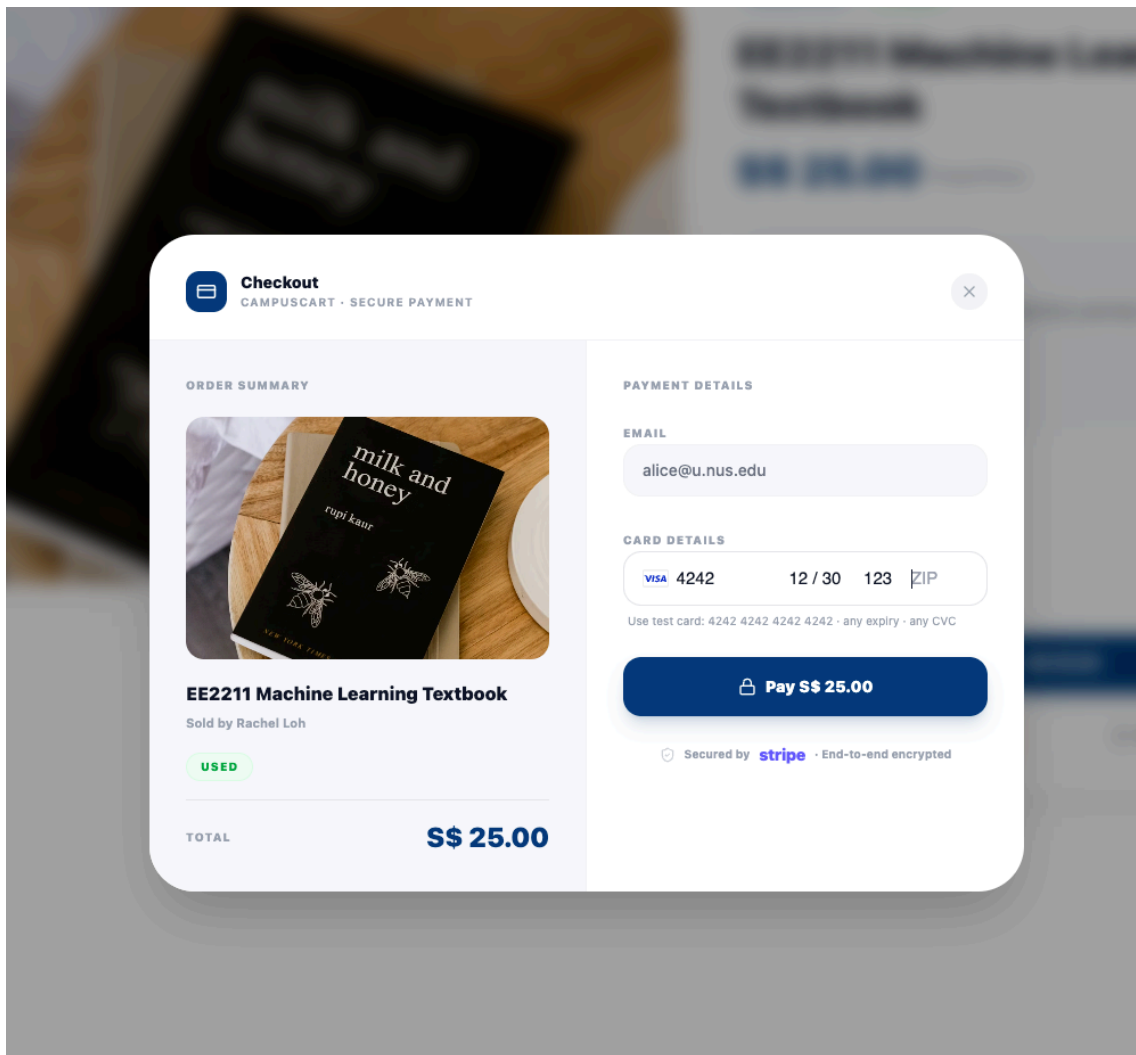


Figure 3.6.1: Stripe payment modal — secure checkout with order summary and embedded card form.

### 3.7 Performance, Caching & Dev Experience

- **Shopify Integration:** External product data is fetched and cached in `sessionStorage` (5-min TTL, `src/services/shopifyService.js`).
- **Developer Experience:** One-command setup for evaluators ( `docker-compose.yml` ), hot reload for developers ( `docker-compose.dev.yml` ). Linting, Prettier, Husky hooks, and CI/CD via GitHub Actions ensure code quality and consistency.
- **Documentation:** `README.md` (developer guide) and `resources/DEMO.md` (evaluator guide) provide onboarding and troubleshooting.

---

## 4. Software Engineering Practices

CampusCart was developed with a strong emphasis on software engineering best practices (*for a deeper dive into the specific tools and configurations used to enforce these standards, please refer to **Section 7: Frameworks, Tooling & Developer Experience***):

- **Documentation:** All major functions and modules are commented. Onboarding and troubleshooting are covered in the README and DEMO.md.
  - **Usability:** All forms have validation, error messages, and are mobile-friendly. Navigation is intuitive, with clear CTAs and feedback.
  - **Modularization:** React components, GraphQL resolvers, and services are modular and reusable. Shared constants for categories/locations ensure consistency.
  - **Code Originality:** All code is original except where noted in README. Any borrowed code is attributed.
  - **Automation:** Setup scripts for DB, seeding, and Docker. Lint/format on commit. CI/CD pipeline for PRs.
  - **Testing & Quality:** Husky git hooks enforce branch naming and prevent direct pushes to main. All team members work via Pull Requests. GitHub Actions run linting, formatting, and build checks on every PR.
- 

## 5. AI Integration & Model Selection

A key innovation in CampusCart is the integration of local, privacy-preserving AI to enhance the user experience. We implemented two primary AI-driven features: image-to-text autofill for listing creation and intelligent search suggestions. This was achieved using a locally-hosted Large Language Model (LLM) via the Ollama framework.

### 5.1 AI Feature Workflow

The primary AI feature, "Auto-Fill," streamlines the listing creation process. The workflow is as follows:

1. **Client-Side Trigger:** The user clicks the "Auto-Fill with AI" button and selects an image of their item from their device ( `src/components/ui/AiAutoFillButton.jsx` ).
2. **Image Upload:** The frontend sends the image file to a dedicated REST endpoint on our Express server ( `server/routes/ai.js` ). We use a REST endpoint here as GraphQL is not optimized for multipart file uploads.
3. **Backend Processing:** The server receives the image and forwards it along with a structured prompt to the Ollama service, which runs the vision-capable LLM ( `server/services/ai.js` ). The prompt instructs the model to analyze the image and return a JSON object containing a suggested `title` , `description` , `category` , `condition` , and estimated `price` .
4. **LLM Inference:** The `llava:7b` model, running within Ollama, processes the image and prompt, generating the structured JSON output.
5. **Response & Form Population:** The backend service parses the model's response, validates the JSON, and sends it back to the frontend. The React client then uses this data to populate the fields of the "Create Listing" form, saving the user significant time and effort.

This entire process is designed to be fast and seamless, providing a "magical" user experience while maintaining full user control, as they can edit any of the AI-generated fields before submitting.

### 5.2 Model Selection Rationale

Choosing the right model was critical and involved a trade-off analysis across several key criteria. Our primary constraint was the desire to run the model locally to ensure privacy, eliminate API costs, and reduce network latency. We evaluated several open-source vision models based on the following:

- **Cost & Accessibility:** Our goal was zero cost. Cloud-based vision APIs like GPT-4V or Google Gemini are powerful but incur costs per API call, making them unsuitable for an academic project and posing a barrier to future open-source adoption. Local models run via Ollama are free to use.

- **Latency:** Network latency to external APIs can create a sluggish user experience. By running the model locally within our Docker Compose setup, inference happens on the same machine, reducing round-trip time to just the local processing duration.
- **Accuracy & Capability:** The model needed to be a multi-modal (vision) LLM capable of analyzing an image and generating structured JSON output. We tested several models, including `llava:7b`, `bakllava`, and others. `llava:7b` demonstrated a superior ability to correctly identify common student-related items (textbooks, electronics, furniture), describe them accurately, and provide reasonable price estimates within the Singaporean context.
- **Resource Consumption (Model Size):** Local models require significant RAM and CPU/GPU resources. We needed a model that was powerful enough for the task but small enough to run effectively on typical developer hardware (e.g., a modern laptop with 16GB of RAM) without requiring specialized GPU hardware. The 7-billion parameter `llava:7b` model offered the best balance, providing good performance without the prohibitive resource requirements of larger 13B or 34B models.

**Conclusion:** The `llava:7b` model, served via Ollama, was the optimal choice for CampusCart. It provided the best trade-off between cost, performance, accuracy, and resource requirements, enabling us to deliver advanced AI features in a privacy-preserving and economically sustainable manner.

---

## 6. Competition, Legal, and Business Model

- **Competition:** See Section 1 for detailed analysis. CampusCart is unique in the NUS context, with features not found in Carousell or Telegram groups.
  - **Legal:** All dependencies are open source. No proprietary code. No personal data stored beyond demo/test accounts. Project is open source and ready for future extension or commercialization.
  - **Business Model:** While this is an academic project, the architecture supports future upgrades (real payments, mobile app, OAuth login, production deployment). The codebase is modular and well-documented for handover or open-source adoption.
- 

## 7. Frameworks, Tooling & Developer Experience

CampusCart goes beyond standard academic expectations by implementing robust engineering workflows and tooling to ensure code quality, developer productivity, and maintainability:

- **Husky Git Hooks** (<https://typicode.github.io/husky/>): We use Husky to automate quality checks at critical stages of the development lifecycle. Before any code can be committed, Husky automatically runs Prettier and ESLint to format the code and check for errors. This prevents inconsistent or broken code from ever entering the version history. It also enforces branch naming conventions, ensuring our Git history remains clean and easy to navigate. This practice is standard in professional software teams to maintain high codebase quality.
- **Prisma ORM** (<https://www.prisma.io/>): Instead of writing raw SQL queries, which are error-prone and hard to maintain, we adopted Prisma as our Object-Relational Mapper. Prisma allows us to define our database schema in a simple, declarative file. It then auto-generates a fully type-safe client that we use to query the database in our backend code.
  - **Key Benefits:** This provides compile-time error checking, autocompletion in our IDEs, and prevents entire classes of bugs like SQL injection. It also handles database migrations automatically, making schema changes safe and repeatable across all developer machines and our production environment. For a project of this scale, Prisma was instrumental in accelerating local development while ensuring data integrity through ORM synchronization.

- **GitHub Workflows (CI/CD):** We configured automated Continuous Integration (CI) pipelines using GitHub Actions. On every Pull Request, these workflows automatically run a series of checks—including linting, formatting, and build tests—to ensure that proposed changes meet our quality standards and do not introduce regressions. This automated gatekeeping is essential for collaborative projects, as it provides immediate feedback and maintains the health of our main codebase.
- **Prettier & ESLint:** These tools are the foundation of our code quality strategy. ESLint analyzes our code to find and fix stylistic or programmatic problems, while Prettier enforces a consistent code style across the entire project. By automating this, we eliminate debates over formatting and can focus entirely on the logic and functionality of the application.
- **Docker Compose:** Provides one-command setup for both evaluators and developers, ensuring environment parity and easy onboarding.
- **Comprehensive Documentation:** All setup, troubleshooting, and onboarding steps are documented in `README.md` and `resources/DEMO.md`.

These frameworks and tools make the project more robust, maintainable, and ready for real-world collaboration or handover.

---

## 8. Future Scalability & Deployment

CampusCart is designed with extensibility and real-world deployment in mind. Several architectural choices make it easy to scale, migrate to cloud infrastructure, or add new features with minimal refactoring:

- **Cloud Image Storage:** While images are currently stored on the local server filesystem, the upload pipeline is abstracted so that switching to a cloud provider (e.g., AWS S3, Google Cloud Storage) only requires changing the upload endpoint and updating the image URLs. The rest of the application is storage-agnostic.
- **Containerization:** All services (frontend, backend, database, AI model) are containerized via Docker Compose, making it straightforward to deploy to any cloud provider supporting containers (AWS ECS, Azure, GCP, etc.).
- **Database Scalability:** PostgreSQL is production-ready and can be migrated to managed services (e.g., AWS RDS, Azure Database for PostgreSQL) with no code changes.
- **AI Model Flexibility:** The AI integration is modular. The Ollama service can be swapped for a cloud-based LLM or upgraded to a more powerful local model as hardware or budget allows.
- **Payments:** Stripe integration is ready for production. Switching from test to live mode is a single environment variable change. Adding webhook support for robust order status updates is straightforward.
- **Authentication:** The current email-based authentication can be extended to support OAuth (Google, NUSNET, etc.) for broader access and easier onboarding.
- **Mobile & PWA:** The frontend is built with responsive design and can be further enhanced as a Progressive Web App (PWA) or ported to React Native for a native mobile experience.
- **DevOps & CI/CD:** The project already uses GitHub Actions for CI. It can be extended to support automated deployment pipelines to cloud platforms.
- **Feature Roadmap:**
  - Semantic search and recommendations using vector databases (e.g., pgvector)
  - Real-time chat and notifications
  - Advanced analytics for sellers
  - Multi-campus or multi-tenant support



These design decisions ensure that CampusCart can grow from an academic prototype to a robust, production-ready platform with minimal technical debt.

## Appendix A — Prototype Mockups (Initial Figma designs)

The original Figma prototypes used during the design phase are reproduced below for reference. These mockups were used to align the team on layout, component behaviour, and information architecture prior to implementation.

### Authentication & Entry Point

The login screen establishes CampusCart's identity as an NUS-exclusive platform from the very first interaction. The prominent "Sign in with NUS Email" call-to-action and the "Exclusively for the NUS Community" badge immediately communicate trust and exclusivity.

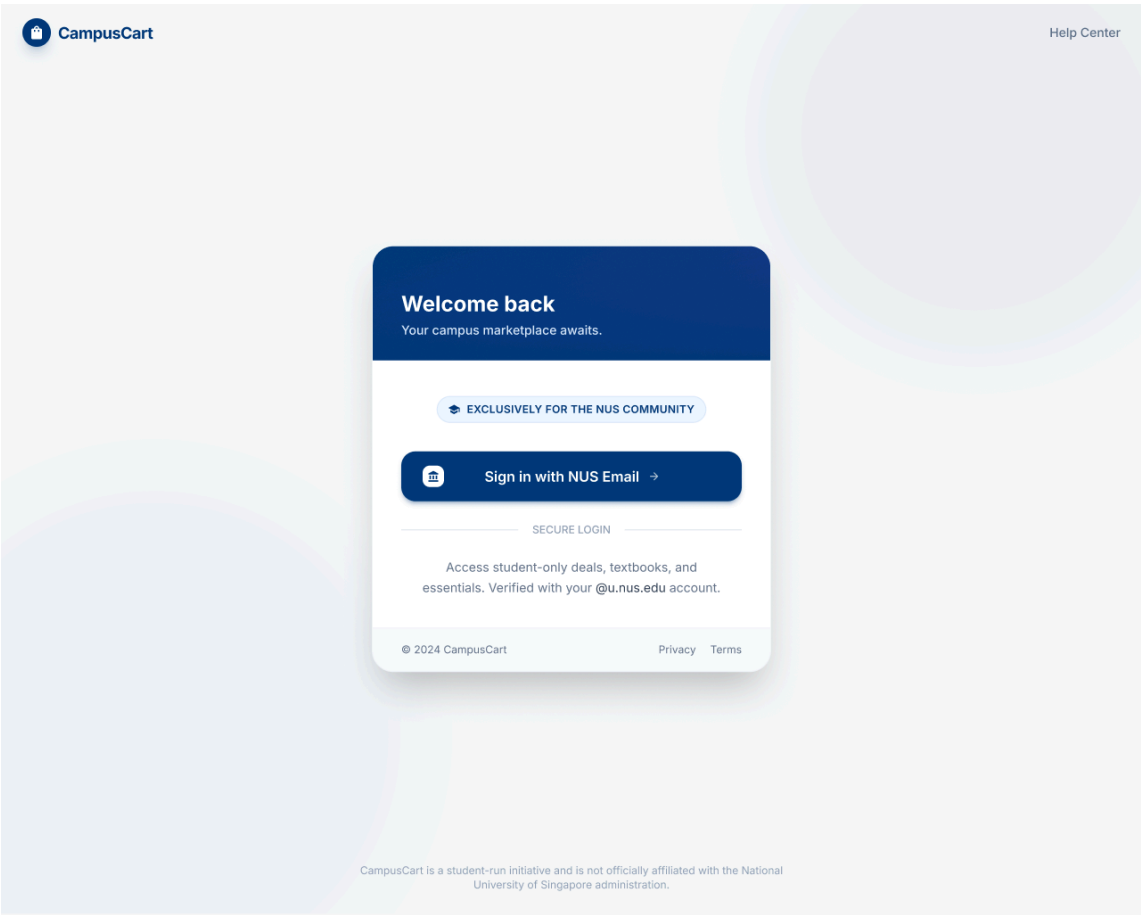


Figure A.1: Login page prototype — emphasizing NUS-exclusive access and verified student identity.

### Marketplace Browse & Discovery

The homepage serves as the primary discovery surface. The prototype establishes the core browsing experience: a hero search bar, category quick-filters (Textbooks, Furniture, Electronics, Clothing), and a card-based listing grid with condition badges, pricing, location tags, and seller verification status. A sidebar provides advanced filtering by condition, price range, and campus location.

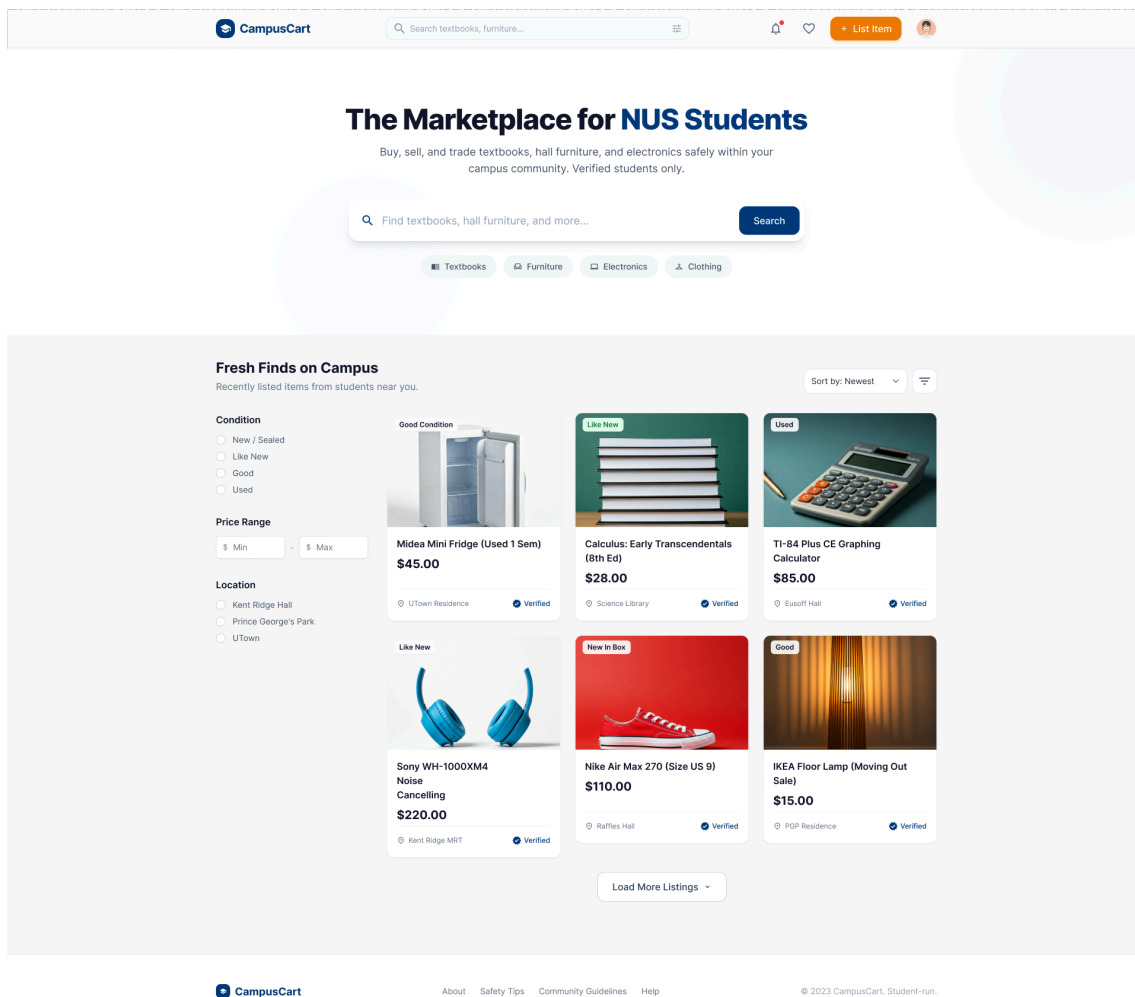


Figure A.2: Marketplace homepage prototype — hero search, category filters, and listing cards with condition/location metadata.

## Product Detail & Seller Interaction

The product detail view was designed to give buyers all the information they need to make a purchase decision without leaving the page. It features a multi-image gallery, structured item metadata (price, condition, category), a detailed text description, and a seller information card showing verification status and response time. The prominent "Make an Offer" and "Chat with Seller" buttons streamline the transaction initiation flow.

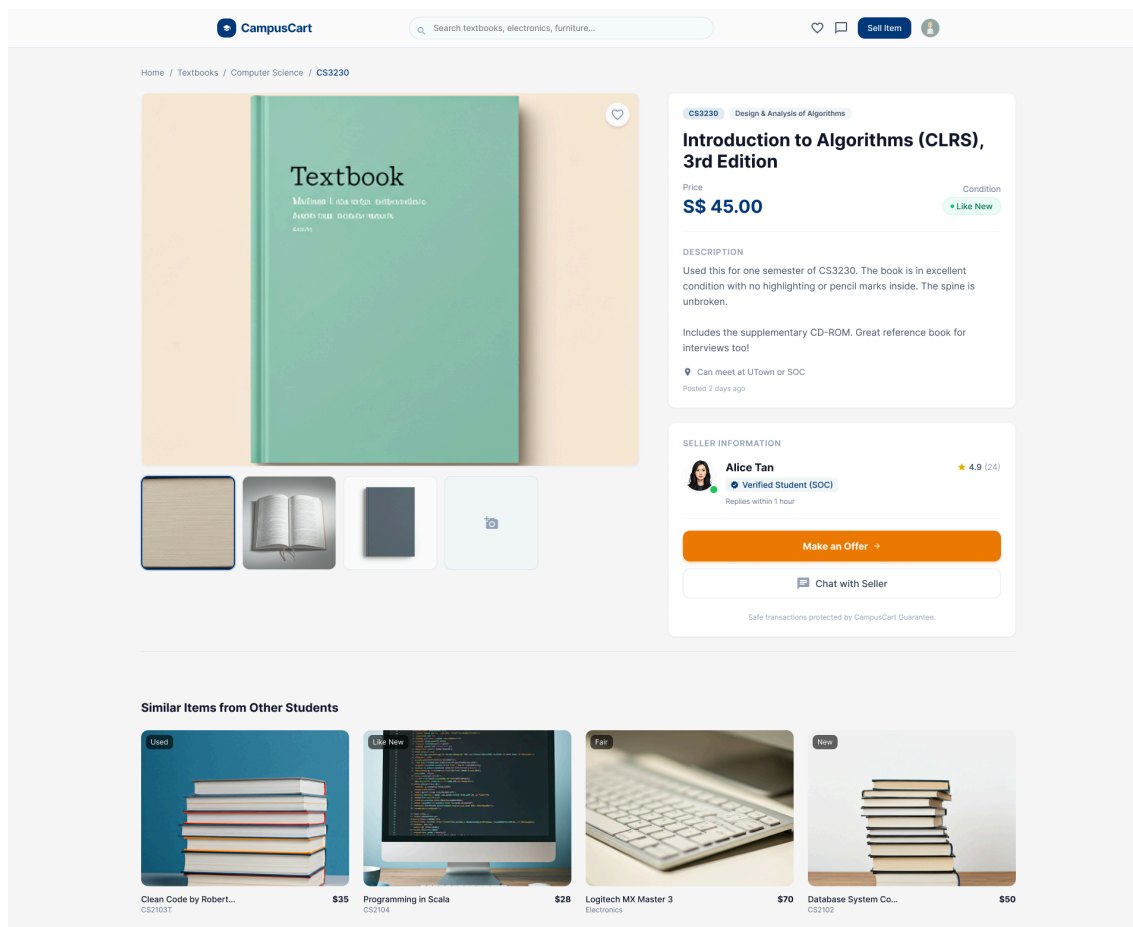


Figure A.3: Product detail prototype — image gallery, item metadata, seller card, and direct offer/chat actions.

## Seller Dashboard & Listing Management

The seller dashboard provides a comprehensive management interface with at-a-glance analytics (active listings count, total earnings, item views), tabbed navigation for active/sold/draft listings, and inline actions (edit, mark sold) for each item. This design informed the GraphQL mutation structure for listing lifecycle management.

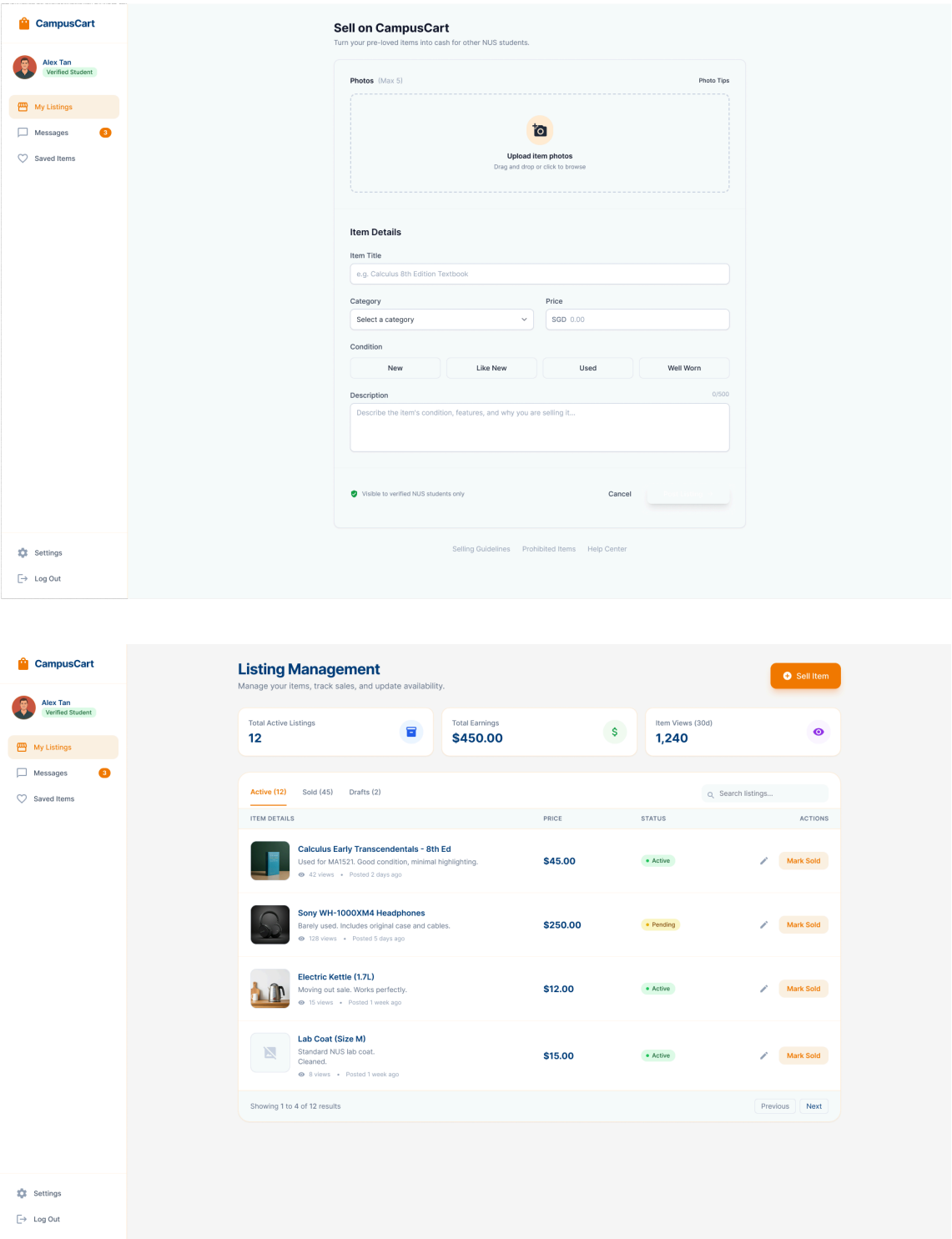


Figure A.4 (left): Listing creation form with photo upload, category, condition selector, and pricing. Figure A.5 (right): Seller dashboard with listing management, analytics, and status tracking.

### Saved Items & Account Settings

The Saved Items view allows buyers to bookmark listings for later consideration, while the Account Settings page provides comprehensive profile management including personal information, contact details,

notification preferences, and account lifecycle controls.

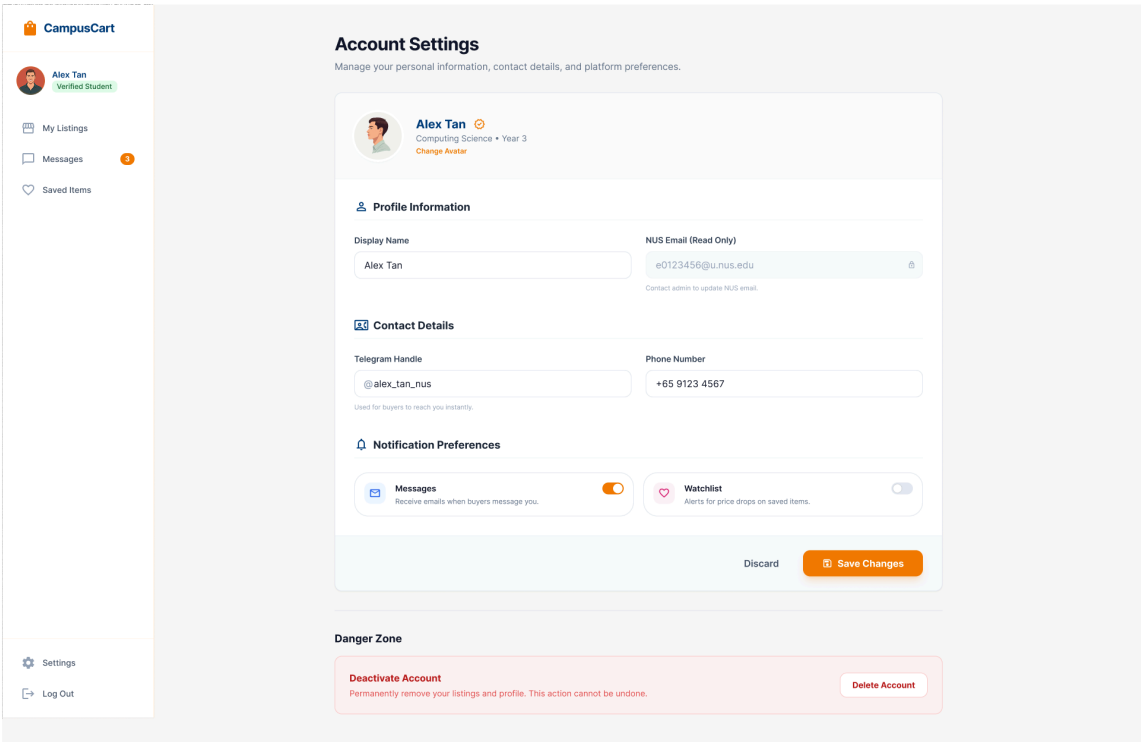
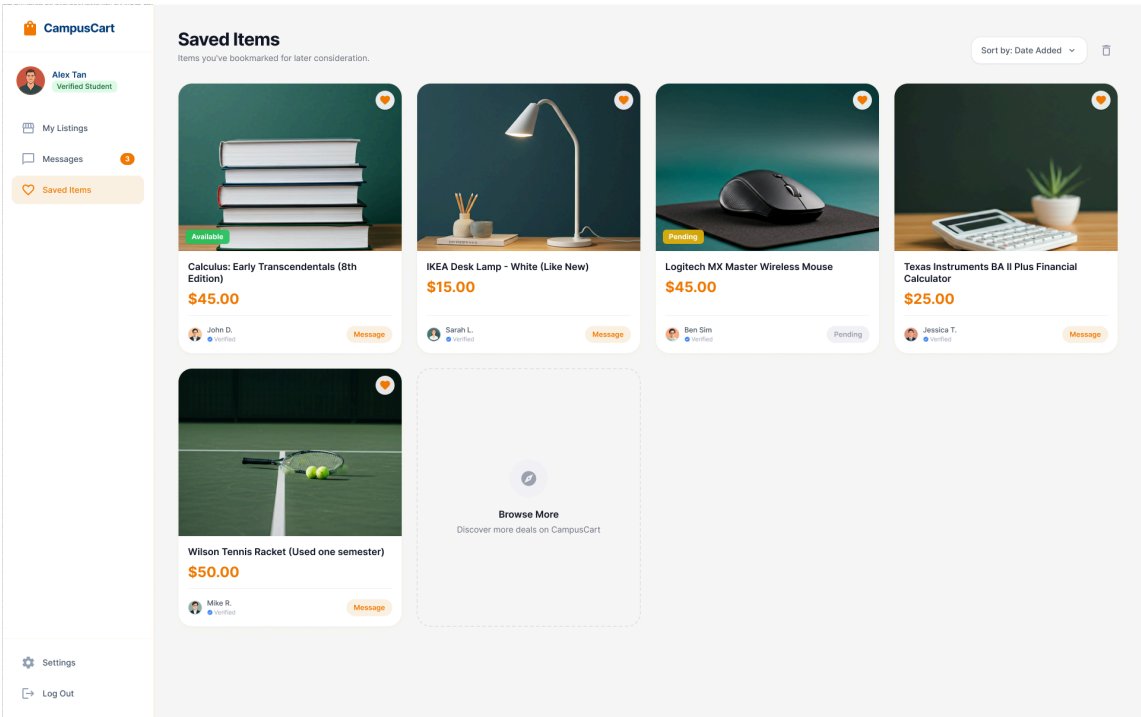


Figure A.6 (left): Saved items view with bookmarked listings and availability status. Figure A.7 (right): Account settings with profile information, contact details, and notification preferences.

*Note: These prototypes informed the implemented UI. For the formal report we moved the full prototype documentation to this appendix so the main body can focus on the delivered features.*